DASTURLASH 1 • DAS1110

6-Ma'ruza: Belgi, Satrlar va Metodlar

Belgi (char) va satr (string) bilan ishlash, formatlash. Metodlar: e'lon qilish, parametrlar, params, rekursiya, ref/out.

O'zbekiston Milliy universiteti Jizzax filiali | Amaliy matematika fakulteti

O'qituvchi: Toshpulatov Muhiddin, PhD, dotsent | 2025/2026-o'quv yili

O'tgan darsni eslaymiz

- Massiv — bir xil turdagi bir nechta qiymatni BITTA nom ostida saqlovchi tuzilma; indekslar 0 dan boshlanadi.
- foreach — massiv elementlarini indekssiz, ketma-ket o'qish uchun qulay.
- Random klassi yordamida massivni tasodifiy sonlar bilan to'ldirish mumkin.
- Bugun ikkita katta mavzuni ko'ramiz: BELGI/SATRLAR bilan ishlash va o'z METODLARIMIZNI yozish — bular bilan kod ancha tartibli va qayta ishlatiladigan bo'ladi.

Bugungi darsning rejasi (katta hajmli dars!)

- I-BO'LIM: Belgi (char) turi va funksiyalari
- II-BO'LIM: Belgilar massivi va satrlar ustida amallar
- III-BO'LIM: Satrlarni formatlash
- IV-BO'LIM: Metodlar bilan ishlash — e'lon qilish, parametrlar
- V-BO'LIM: Metodlarning qo'shimcha imkoniyatlari — params, rekursiya
- VI-BO'LIM: Qiymat/havola turlari, ref va out kalit so'zlari

MAQSAD

Dars yakunida siz nimalarga erishasiz?

- char turi ustida funksiyalardan (IsDigit, IsLetter, ToUpper va h.k.) foydalana olasiz.
- Satrlar (string) ustida asosiy amallarni (Length, Substring, Split, Replace) bajara olasiz.
- Sonlar va satrlarni formatlab chiqara olasiz (string interpolation, format specifiers).
- O'z metodlaringizni parametrlar va qaytariladigan qiymat bilan yoza olasiz.
- params, metodlarni qayta yuklash, rekursiya va ref/out kalit so'zlaridan foydalana olasiz.

I

1-BO'LIM

Belgi (char) turi va funksiyalari

Bitta belgi ustida nimalar qila olishimizni ko'ramiz.

char turi ustidagi foydali funksiyalar

- char — bitta belgini saqlovchi tur edi (o'tgan darslarda ko'rgan edik); C# unga tayyor FUNKSIYALAR (metodlar) bilan birga beradi.
- char.IsDigit(belgi) — belgi raqam (0-9) bo'lsa true qaytaradi.
- char.IsLetter(belgi) — belgi harf bo'lsa true qaytaradi.
- char.IsUpper(belgi) / char.IsLower(belgi) — belgi katta/kichik harf ekanini tekshiradi.
- char.ToUpper(belgi) / char.ToLower(belgi) — belgini katta/kichik harfga o'giradi.

Misol: char funksiyalari

```
char belgi = 'a';
Console.WriteLine(char.IsLetter(belgi));
Console.WriteLine(char.IsDigit(belgi));
Console.WriteLine(char.ToUpper(belgi));
```

► NATIJA (Console chiqishi):

```
True
False
A
```

🧠 VIKTORINA #1

char.IsDigit('5') chaqirilsa, natija qanday bo'ladi?

A char.IsDigit('5') — true qaytaradi, chunki '5' raqam belgisi

B char.IsDigit('5') — false qaytaradi, chunki '5' satrli(char turidagi) o‘zgaruvchi sifatida yoziladi

C char.IsDigit faqat int turi bilan ishlaydi

D char.IsDigit('5') — xato beradi

O'ylab ko'ring... javobingizni chatga yozing!

✅ JAVOB #1

char.IsDigit('5') chaqirilsa, natija qanday bo'ladi?

✓ **char.IsDigit('5') — true qaytaradi, chunki '5' raqam belgisi**

B char.IsDigit('5') — false qaytaradi, chunki '5' satr sifatida yoziladi

C char.IsDigit faqat int turi bilan ishlaydi

D char.IsDigit('5') — xato beradi

💡 Izoh: '5' bitta tirnoqda yozilgan -- bu char literali, raqam belgisi. char.IsDigit shu belgi 0-9 oralig'ida ekanini tekshiradi va true qaytaradi.

II

2-BO'LIM

Belgilar massivi va satrlar ustida amallar

Satr aslida belgilar ketma-ketligi ekanini va u bilan ishlash usullarini ko'ramiz.

Satr (string) va belgilar massivi (char[])

- Satr (string) — aslida BELGILAR KETMA-KETLIGI (char[ ] ning maxsus, qulayroq shakli).
- Satrning har bir belgisiga INDEKS orqali murojaat qilish mumkin: satr[0] — birinchi belgi.
- Satrni belgilar massiviga aylantirish: char[ ] belgilar = satr.ToCharArray();
- MUHIM: C# da string O'ZGARMAS (immutable) — satr[0] = 'A'; kabi YOZISH MUMKIN EMAS, satrni faqat BUTUNLAY yangilash mumkin.

Satrlar ustida eng ko'p ishlatiladigan amallar

Metod / xossa	Vazifasi	Misol
Length	Satr uzunligini qaytaradi	"Salom".Length // 5
Substring(i, n)	i indeksdan n ta belgini ajratib oladi	"Salom".Substring(0, 2) // "Sa"
IndexOf(s)	s qism-satr qaysi indeksda boshlanishini topadi	"Salom".IndexOf("lo") // 2
Contains(s)	Satr ichida s bor-yo'qligini tekshiradi	"Salom".Contains("al") // true
Replace(a, b)	a ni b ga almashtiradi	"Salom".Replace("a", "o") // "Solom"
Split(belgi)	Satrni belgi bo'yicha bo'laklarga ajratadi	"a,b,c".Split(',') // “a”, “b”, “c”
Trim()	Boshi/oxiridagi bo'sh joylarni olib tashlaydi	" Salom ".Trim() // "Salom"
ToUpper() / ToLower()	Katta/kichik harfga o'giradi	"Salom".ToUpper() // "SALOM"

Misol: satr amallarini birgalikda qo'llash

```
string ism = "  Aziza Karimova  ";
string toza = ism.Trim();
Console.WriteLine(toza.Length);
Console.WriteLine(toza.ToUpper());
string[] qismlar = toza.Split(' ');
Console.WriteLine(qismlar[0]);
```

► NATIJA (Console chiqishi):

```
14
```

► NATIJA (Console chiqishi):

```
AZIZA KARIMOVA
```

► NATIJA (Console chiqishi):

```
Aziza
```

🧠 VIKTORINA #2

C# da string turi haqida qaysi gap TO'G'RI?

A "Salom"[0] = 'X'; -- satrning birinchi belgisini o'zgartiradi

B string turi o'zgarmas (immutable) -- satr[0] = 'X'; kabi yozish MUMKIN EMAS

C string va char[] bir xil, farqi yo'q

D Satrlarni hech qachon indeks bilan o'qib bo'lmaydi

O'ylab ko'ring... javobingizni chatga yozing!

✅ JAVOB #2

C# da string turi haqida qaysi gap TO'G'RI?

A "Salom"[0] = 'X'; -- satrning birinchi belgisini o'zgartiradi

✓ **string turi o'zgarmas (immutable) -- satr[0] = 'X'; kabi yozish MUMKIN EMAS**

C string va char[] bir xil, farqi yo'q

D Satrlarni hech qachon indeks bilan o'qib bo'lmaydi

💡 Izoh: string -- o'zgarmas (immutable) tur: mavjud satrning bitta belgisini to'g'ridan-to'g'ri yozib bo'lmaydi, faqat YANGI satr yaratish mumkin (masalan Replace() orqali).

III

3-BO'LIM

Satrlarni formatlash

Sonlar, sanalar va o'zgaruvchilarni chiroyli ko'rinishda chiqarishni o'rganamiz.

String interpolation va formatlash

- String interpolation: $"Ism: {ism}, Yosh: {yosh}" — o'zgaruvchilarni to'g'ridan-to'g'ri matn ichiga qo'yish usuli (oldingi darsda ko'rgan edik).
- Sonni formatlab chiqarish: narx.ToString("N2") — 2 xonali kasr bilan (masalan 1234.5 -> "1,234.50").
- Format specifierlar: N — son (minglik ajratgich bilan), C — valyuta, P — foiz, D — butun son belgilangan uzunlikda.
- Sana formatlash: DateTime.Now.ToString("dd.MM.yyyy") — joriy sanani kun.oy.yil ko'rinishida chiqaradi.

Misol: son va o'zgaruvchilarni formatlash

```
double narx = 1234.567;
Console.WriteLine(narx.ToString("N2"));
string ism = "Aziza";
int yosh = 20;
Console.WriteLine($"Ism: {ism}, Yosh: {yosh}");
```

► NATIJA (Console chiqishi):

```
1,234.57
```

► NATIJA (Console chiqishi):

```
Ism: Aziza, Yosh: 20
```

O'zgaruvchan satrlar: StringBuilder

- string o'zgarmas bo'lgani uchun, siklda KO'P marta satrni "qo'shib" borish (masalan natija += so'z;) xotirani SAMARASIZ ishlatadi — har safar YANGI satr yaratiladi.
- StringBuilder — maxsus, O'ZGARUVCHAN (mutable) satr klassi; ko'p marta o'zgartirish kerak bo'lganda ancha SAMARALIROQ.
- Foydalanish: StringBuilder sb = new StringBuilder(); so'ng sb.Append("matn"); deb qo'shib boriladi.
- Oxirida sb.ToString() orqali oddiy string ga aylantiriladi.

💡 *Boshlang'ich bosqichda oddiy string + yetarli; StringBuilder katta hajmli matn bilan ishlaganda foydali bo'ladi.*

🧠 VIKTORINA #3

narx.ToString("N2") format turi nima qiladi?

A "N2" -- 2 ta butun son qo'shimcha qo'shadi

B "N2" -- sonni 2 xonali kasr qism bilan, minglik ajratgich qo'yib formatlaydi

C "N2" -- sonni 2 marta ko'paytiradi

D "N2" -- faqat sanalar uchun ishlatiladi

O'ylab ko'ring... javobingizni chatga yozing!

✅ JAVOB #3

narx.ToString("N2") format specifiri nima qiladi?

A "N2" -- 2 ta butun son qo'shimcha qo'shadi

✓ **"N2" -- sonni 2 xonali kasr qism bilan, minglik ajratgich qo'yib formatlaydi**

C "N2" -- sonni 2 marta ko'paytiradi

D "N2" -- faqat sanalar uchun ishlatiladi

💡 Izoh: "N2" -- sonni 2 xonali kasr qismi bilan va minglik xonalarini vergul bilan ajratib formatlaydi (masalan 1234.567 -> "1,234.57").

IV

4-BO'LIM

Metodlar bilan ishlash

Kodni qismlarga bo'lib, qayta ishlatish mumkin bo'lgan bloklarga aylantirishni o'rganamiz.

Metod (method) nega kerak?

- Agar bir xil kod BIR NECHA joyda takrorlansa, uni har safar qayta yozish o'rniga, BITTA metodga joylab, kerak bo'lganda CHAQIRISH mumkin.
- Metod — ma'lum bir vazifani bajaruvchi, NOM berilgan kod bo'lagi (aslida Main() ham — bu metod!).
- Afzalliklari: kod TAKRORLANMAYDI (DRY — Don't Repeat Yourself), dastur TUZILGAN va TUSHUNARLI bo'ladi, xatoni topish OSONLASHADI.

Metod e'lon qilish sintaksisi

- Sintaksisi: modifikator qaytariladigan_tur MetodNomi(parametrlar) { // kod }
- Masalan: static void Salomla() { Console.WriteLine("Salom!"); }
- static — hozircha barcha metodlarimiz shu kalit so'z bilan yoziladi (Main() singari); buning sababini keyingi darslarda chuqurroq ko'ramiz.
- void — metod HECH QANDAY qiymat QAYTARMASLIGINI bildiradi.
- Metodni CHAQIRISH uchun uning nomini yozish kifoya: Salomla();

Misol: parametrsiz, qiymat qaytarmaydigan metod

```
static void Salomla()
{
    Console.WriteLine("Salom!");
}

// Main() ichida chaqiramiz:
Salomla();
Salomla();
```

► NATIJA (Console chiqishi):

```
Salom!
```

► NATIJA (Console chiqishi):

```
Salom!
Salom!
```

Bitta metodni xohlagancha marta chaqirish mumkin -- kodni qayta yozishning hojati yo'q.

Parametrli metodlar

- Parametr — metodga TASHQARIDAN ma'lumot uzatish uchun ishlatiladigan o'zgaruvchi.
- Sintaksisi: static void MetodNomi(tur parametrNomi) { ... }
- Bir nechta parametr vergul bilan ajratiladi: static void Salomla(string ism, int yosh) { ... }
- Chaqirishda parametr qiymatlari ARGUMENT deb ataladi: Salomla("Aziza", 20);

Misol: bitta parametrli metod

```
static void Salomla(string ism)
{
    Console.WriteLine($"Salom, {ism}!");
}

Salomla("Aziza");
Salomla("Jasur");
```

► NATIJA (Console chiqishi):

```
Salom, Aziza!
```

► NATIJA (Console chiqishi):

```
Salom, Aziza!
Salom, Jasur!
```

Metoddan qiymat qaytarish

- Agar metod natija QAYTARISHI kerak bo'lsa, void o'rniga QAYTARILADIGAN TUR yoziladi (masalan int, string, bool).
- return kalit so'zi — metoddan qiymatni qaytaradi VA metodni shu yerda TUGATADI.
- Qaytarilgan qiymatni o'zgaruvchiga saqlash yoki to'g'ridan-to'g'ri ishlatish mumkin.

Misol: qiymat qaytaruvchi metod

```
static int Yigindi(int a, int b)
{
    return a + b;
}

int natija = Yigindi(5, 7);
Console.WriteLine(natija);
```

► NATIJA (Console chiqishi):

```
12
```

Kelishuv bo'yicha parametrli metodlar (default parameters)

- Parametrga STANDART (kelishuv bo'yicha) qiymat berish mumkin — chaqirishda argument berilmasa, shu qiymat ishlatiladi.
- Sintaksisi: static void Salomla(string ism = "Mehmon") { ... }
- Chaqirishda argument berish IXTIYORIY bo'ladi: Salomla(); // "Mehmon" ishlatiladi, Salomla("Aziza"); // "Aziza" ishlatiladi.
- QOIDA: kelishuv bo'yicha parametrlar har doim parametrlar RO'YXATINING OXIRIDA bo'lishi kerak.

🧠 VIKTORINA #4

Metod e'lon qilishda void bilan int orasidagi farq nima?

A void -- metod qiymat qaytarmasligini bildiradi; int/string va h.k. -- metod shu turdagi qiymat qaytarishini bildiradi

B void va int bir xil, farqi yo'q

C void faqat parametrsiz metodlarda ishlatiladi

D return kalit so'zi faqat void metodlarda ishlatiladi

O'ylab ko'ring... javobingizni chatga yozing!

✅ JAVOB #4

Metod e'lon qilishda void bilan int orasidagi farq nima?

✓ **void -- metod qiymat qaytarmasligini bildiradi; int/string va h.k. -- metod shu turdagi qiymat qaytarishini bildiradi**

B void va int bir xil, farqi yo'q

C void faqat parametrsiz metodlarda ishlatiladi

D return kalit so'zi faqat void metodlarda ishlatiladi

💡 Izoh: void -- metod hech qanday qiymat qaytarmaydi. int, string, bool va h.k. -- metod chaqiruvchiga shu turdagi natija qaytarishi SHART (return orqali).

V

5-BO'LIM

Metodlarning qo'shimcha imkoniyatlari

Massiv parametrlar, params, qayta yuklash va rekursiya bilan tanishamiz.

Metodlarda massivlardan foydalanish

- Massivni metodga PARAMETR sifatida uzatish mumkin — bu butun massiv ustida ishlovchi metod yozishga imkon beradi.
- Sintaksisi: static int Yigindi(int[] sonlar) { ... }
- Massiv REFERENCE (havola) turi bo'lgani uchun, metod ichida massiv elementini o'zgartirsa, bu O'ZGARISH chaqirgan joyga ham TA'SIR qiladi!

Misol: massivni parametr sifatida qabul qiluvchi metod

```
static int Yigindi(int[] sonlar)
{
    int jami = 0;
    foreach (int s in sonlar) jami += s;
    return jami;
}

int[] ballar = { 10, 20, 30 };
Console.WriteLine(Yigindi(ballar));
```

► NATIJA (Console chiqishi):

```
60
```

params kalit so'zi

- Ba'zan metodga NECHTA argument berilishi OLDINDAN noma'lum bo'ladi — params shuning uchun ishlatiladi.
- Sintaksisi: static int Yigindi(params int[] sonlar) { ... }
- Chaqirishda massivni QAVSSIZ, vergul bilan ajratib yuborish mumkin: Yigindi(1, 2, 3); yoki Yigindi(1, 2, 3, 4, 5);
- params faqat parametrlar ro'yxatining ENG OXIRIDA, va BITTAGINA bo'lishi mumkin.

Misol: params bilan ixtiyoriy sondagi argument

```
static int Yigindi(params int[] sonlar)
{
    int jami = 0;
    foreach (int s in sonlar) jami += s;
    return jami;
}

Console.WriteLine(Yigindi(1, 2));
Console.WriteLine(Yigindi(1, 2, 3, 4));
```

► NATIJA (Console chiqishi):

```
3
```

► NATIJA (Console chiqishi):

```
3
10
```

Bir xil metod ham 2, ham 4 ta argument bilan chaqirildi -- params shuni qulaylashtiradi.

Metodlarni qayta yuklash (overloading)

- Qayta yuklash — BIR XIL nomli, lekin PARAMETRLARI (soni yoki turi) boshqacha bo'lgan bir nechta metod yaratish imkoni.
- Masalan: int Yigindi(int a, int b) va double Yigindi(double a, double b) — ikkalasi ham "Yigindi" deb ataladi.
- C# qaysi metodni chaqirishni argumentlarning TURI va SONIGA qarab avtomatik ANIQLAYDI.
- Bu — Console.WriteLine() ning turli argumentlar bilan (son, satr, bool) ishlay olishining aynan shu mexanizm tufayli ekanini bildiradi!

Rekursiv metodlar

- Rekursiya — metodning O'Z-O'ZINI chaqirishi.
- Har bir rekursiv metod ikkita qismga ega bo'lishi SHART: BAZA HOLATI (qachon to'xtash kerakligi) va REKURSIV CHAQIRUV (o'zini kichikroq masala bilan chaqirish).
- Baza holati bo'lmasa — CHEKSIZ rekursiya yuzaga keladi (StackOverflowException xatosi bilan tugaydi).
- Klassik misol: faktorial hisoblash (5! = 5 * 4 * 3 * 2 * 1).

Misol: faktorialni rekursiya bilan hisoblash

```
static int Faktorial(int n)
{
    if (n <= 1)     // baza holati
        return 1;
    return n * Faktorial(n - 1);  // rekursiv chaqiruv
}

Console.WriteLine(Faktorial(5));
```

► NATIJA (Console chiqishi):

```
120
```

*Faktorial(5) -> 5*Faktorial(4) -> 5*4*Faktorial(3) -> ... -> 5*4*3*2*1 = 120.*

🧠 VIKTORINA #5

Rekursiya haqida qaysi gap TO'G'RI?

A Rekursiv metod hech qachon to'xtamaydi

B Har bir rekursiv metod BAZA HOLATI (to'xtash sharti) ga ega bo'lishi SHART, aks holda cheksiz chaqiruv (StackOverflowException) yuz beradi

C Rekursiya faqat massivlar bilan ishlaydi

D Rekursiv metod o'zini chaqira olmaydi

O'ylab ko'ring... javobingizni chatga yozing!

✅ JAVOB #5

Rekursiya haqida qaysi gap TO'G'RI?

A Rekursiv metod hech qachon to'xtamaydi

✓ **Har bir rekursiv metod BAZA HOLATI (to'xtash sharti) ga ega bo'lishi SHART, aks holda cheksiz chaqiruv (StackOverflowException) yuz beradi**

C Rekursiya faqat massivlar bilan ishlaydi

D Rekursiv metod o'zini chaqira olmaydi

💡 Izoh: Rekursiv metod o'zini chaqiradi, lekin BAZA HOLATI bo'lishi SHART -- aks holda chaqiruvlar cheksiz davom etib, StackOverflowException xatosiga olib keladi.

VI

6-BO'LIM

Qiymat/havola turlari, ref va out

Metodga parametr qanday uzatilishini chuqurroq ko'ramiz.

Qiymat turlari va havola turlari (chuqurroq)

- Qiymat turlari (int, bool, double va h.k.) metodga uzatilganda — QIYMATNING NUSXASI uzatiladi; metod ichidagi o'zgarish TASHQARIGA ta'sir QILMAYDI.
- Havola turlari (massiv, keyingi darslarda o'rganiladigan klasslar) uzatilganda — xotiradagi MANZIL (havola) uzatiladi; metod ichidagi o'zgarish TASHQARIGA ham TA'SIR QILADI (buni V-bo'limda massiv misolida ko'rdik).
- Lekin ba'zan oddiy int/double kabi qiymat turini ham metod ICHIDA o'zgartirib, bu o'zgarishni TASHQARIGA chiqarish kerak bo'ladi — shuning uchun ref va out kalit so'zlari bor.

ref kalit so'zi

- ref — qiymat turini metodga "NUSXA" emas, balki TO'G'RIDAN-TO'G'RI HAVOLA orqali uzatish imkonini beradi.
- Metod ICHIDA qilingan o'zgarish — chaqirgan joydagi o'zgaruvchiga ham TA'SIR QILADI.
- ref bilan ishlashda, parametr METOD E'LONIDA ham, CHAQIRISHDA ham ref so'zi bilan yozilishi SHART.
- MUHIM: ref orqali uzatiladigan o'zgaruvchi chaqirishdan OLDIN biror qiymatga ega bo'lishi kerak.

Misol: ref kalit so'zi bilan qiymatni o'zgartirish

```
static void Ikkilantir(ref int son)
{
    son = son * 2;
}

int x = 5;
Ikkilantir(ref x);
Console.WriteLine(x);  // ref bo'lmasa 5 chiqardi
```

► NATIJA (Console chiqishi):

```
10
```

ref bo'lmaganida x metodga NUSXA sifatida uzatilib, tashqaridagi x hali ham 5 bo'lib qolardi.

out kalit so'zi

- out — ref ga o'xshaydi (havola orqali uzatadi), lekin metodga kirishda o'zgaruvchi OLDINDAN qiymatga ega bo'lishi SHART EMAS.
- out ishlatilgan parametr metod ICHIDA albatta biror qiymat BILAN TO'LDIRILISHI SHART (metoddan chiqishdan oldin).
- Ko'pincha metoddan BIR NECHTA natija qaytarish kerak bo'lganda ishlatiladi (masalan int.TryParse metodida ko'rasiz).

Misol: out bilan BIR NECHTA qiymat qaytarish

```
static void KvadratVaKub(int son, out int kvadrat, out int kub)
{
    kvadrat = son * son;
    kub = son * son * son;
}

KvadratVaKub(3, out int kv, out int kb);
Console.WriteLine($"Kvadrat: {kv}, Kub: {kb}");
```

► NATIJA (Console chiqishi):

```
Kvadrat: 9, Kub: 27
```

Oddiy return orqali metoddan faqat BITTA qiymat qaytarish mumkin -- out shu cheklovni chetlab o'tadi.

🧠 VIKTORINA #6

ref va out kalit so'zlari orasidagi farq nima?

A ref va out bir xil, farqi yo'q

B ref uzatilayotgan o'zgaruvchi OLDINDAN qiymatga ega bo'lishi kerak; out esa metod ICHIDA qiymat bilan to'ldirilishi shart, oldindan qiymat shart emas

C ref faqat massivlar bilan, out faqat satrlar bilan ishlaydi

D Ikkalasi ham faqat qiymat qaytarmaydigan (void) metodlarda ishlatiladi

O'ylab ko'ring... javobingizni chatga yozing!

✅ JAVOB #6

ref va out kalit so'zlari orasidagi farq nima?

A ref va out bir xil, farqi yo'q

✓ **ref uzatilayotgan o'zgaruvchi OLDINDAN qiymatga ega bo'lishi kerak; out esa metod ICHIDA qiymat bilan to'ldirilishi shart, oldindan qiymat shart emas**

C ref faqat massivlar bilan, out faqat satrlar bilan ishlaydi

D Ikkalasi ham faqat qiymat qaytarmaydigan (void) metodlarda ishlatiladi

💡 Izoh: ref bilan uzatiladigan o'zgaruvchi chaqirishdan OLDIN qiymatga ega bo'lishi kerak. out bilan uzatiladigan o'zgaruvchi oldindan qiymatga ega bo'lmasligi mumkin, lekin metod ICHIDA albatta to'ldirilishi SHART.

DIQQAT

Yangi boshlovchilar ko'p qiladigan xatolar

- string[0] = 'X'; kabi satrni to'g'ridan-to'g'ri o'zgartirishga urinish — string o'zgarmas (immutable), bu kompilyatsiya xatosi beradi.
- Metod nomidan keyin qavs ( ) ni unutish — metodni chaqirish uchun HAR DOIM qavs kerak, hatto parametrsiz bo'lsa ham.
- void metoddan qiymat qaytarishga urinish (return qiymat;) — void hech narsa qaytarmasligi kerak.
- Rekursiv metodda BAZA HOLATINI yozishni unutish — bu cheksiz chaqiruv va StackOverflowException ga olib keladi.
- ref/out parametrlarini chaqirishda kalit so'zni yozishni unutish — ular METOD E'LONIDA ham, CHAQIRISHDA ham yozilishi SHART.

Mustaqil bajarish uchun topshiriq

- 1-topshiriq: Kiritilgan satrda nechta unli harf (a, e, i, o, u va o'zbekcha unlilar) borligini hisoblovchi dastur yozing (char.IsLetter va solishtirishdan foydalaning).
- 2-topshiriq: Ikkita son qabul qilib, ularning KATTASINI qaytaruvchi oddiy metod (static int Katta(int a, int b)) yozing.
- 3-topshiriq: params yordamida ixtiyoriy sondagi sonlarning O'RTACHASINI hisoblovchi metod yozing.
- 4-topshiriq (qo'shimcha): Fibonachchi sonini (0,1,1,2,3,5,8...) rekursiya yordamida hisoblovchi metod yozing.

💡 *Har bir metodni avval QOG'OZDA rejalashtiring: nomi, parametrlari, nima qaytaradi -- keyin kodga o'ting.*

Bugungi dars xulosasi

- char.IsDigit/IsLetter/ToUpper kabi funksiyalar belgi ustida tekshirish va o'zgartirish imkonini beradi.
- string — belgilar ketma-ketligi, lekin O'ZGARMAS (immutable); Length, Substring, Split, Replace eng ko'p ishlatiladigan amallar.
- String interpolation ($"...{o'zgaruvchi}...") va ToString("N2") kabi format specifierlar chiqishni chiroyli qiladi.
- Metod — qayta ishlatiladigan kod bo'lagi: parametr qabul qiladi, (ixtiyoriy) qiymat qaytaradi.
- params — ixtiyoriy sondagi argument; rekursiya — metodning o'zini chaqirishi (baza holati SHART).
- ref — oldindan qiymati bor o'zgaruvchini havola orqali uzatadi; out — metod ichida albatta to'ldirilishi kerak bo'lgan natija uchun.

Keyingi mavzular

- 1-semestr lektsiyalarimiz shu bilan YAKUNLANADI
- Keyingi bosqich: amaliy mashg'ulotlar — bugungi va oldingi barcha mavzular bo'yicha mustahkam mashq qilasiz.
- 2-semestrda esa Kortejlar, Obyektga yo'naltirilgan dasturlash (OYD), Sinflar va boshqa chuqur mavzularni ko'ramiz.
- Yakuniy nazoratga tayyorgarlik ko'rishni boshlashni unutmang — barcha 6 ta ma'ruza materialini va viktorinalarni qayta ko'rib chiqing!

Foydali adabiyotlar va havolalar

Darsliklar

- Adambayev U.E. va b. "Algoritmik tillar va dasturlash. C# tilida dasturlash asoslari" (2023)
- Mo'minov B.B. "Dasturlash 1" (2021)
- Albahari B. "C# 7.0. Справочник" (2018)

Onlayn manbalar

- metanit.com/sharp/tutorial — Satrlar va Metodlar bo'limlari
- learn.microsoft.com/dotnet/csharp/programming-guide/classes-and-structs/methods
- www.dastur.uz — o'zbek tilida dasturlash sayti

Rahmat! Butun semestr davomida faol ishtirokingiz uchun!

- 📧 Aloqa uchun: (o'qituvchi email/telefon — dars vaqtida beriladi)
- 📝 Amaliy mashg'ulotlar va mustaqil ishlar — TA bilan davom etadi
- 🗓️ 1-semestr ma'ruzalari yakunlandi — omad tilaymiz!
- 🎯 Faol qatnashganingiz uchun katta rahmat!